

```
import pandas as pd
# basic data exploration
# 1. Load train.csv
df = pd.read_csv("/content/train.csv")
print(df)
```

```

   PassengerId  Survived  Pclass \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3
..          ...         ...     ...
886          887         0       2
887          888         1       1
888          889         0       3
889          890         1       1
890          891         0       3
```

```

      Name      Sex  Age  SibSp \
0      Braund, Mr. Owen Harris    male  22.0     1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0     1
2      Heikkinen, Miss. Laina    female  26.0     0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)    female  35.0     1
4      Allen, Mr. William Henry    male  35.0     0
..          ...         ...     ...
886      Montvila, Rev. Juozas    male  27.0     0
887      Graham, Miss. Margaret Edith    female  19.0     0
888  Johnston, Miss. Catherine Helen "Carrie"    female   NaN     1
889      Behr, Mr. Karl Howell    male  26.0     0
890      Dooley, Mr. Patrick    male  32.0     0
```

```

   Parch  Ticket   Fare Cabin Embarked
0      0   A/5 21171   7.2500   NaN      S
1      0    PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282   7.9250   NaN      S
3      0   113803  53.1000  C123      S
4      0   373450   8.0500   NaN      S
..     ...         ...     ...     ...
886     0   211536  13.0000   NaN      S
887     0   112053  30.0000  B42      S
888     2   W./C. 6607  23.4500   NaN      S
889     0   111369  30.0000  C148      C
890     0   370376   7.7500   NaN      Q
```

[891 rows x 12 columns]

```
# 2. First 10 rows
print(df.head(10))

# 3. Last 10 rows
print(df.tail(10))

# 4. 8 random rows
print(df.sample(8))
```

```

885 female 39.0 0 3 382652 29.1250 NaN Q
886 male 27.0 0 0 211536 13.0000 NaN S
887 female 19.0 0 0 112053 30.0000 B42 S
888 female NaN 1 2 W./C. 6607 23.4500 NaN S
889 male 26.0 0 0 111369 30.0000 C148 C
890 male 32.0 0 0 370376 7.7500 NaN Q

```

	PassengerId	Survived	Pclass	\
678	679	0	3	
779	780	1	1	
881	882	0	3	
364	365	0	3	
534	535	0	3	
765	766	1	1	
811	812	0	3	
801	802	1	2	

	Name	Sex	Age	SibSp	\
678	Goodwin, Mrs. Frederick (Augusta Tyler)	female	43.0	1	
779	Robert, Mrs. Edward Scott (Elisabeth Walton Mc...	female	43.0	0	
881	Markun, Mr. Johann	male	33.0	0	
364	O'Brien, Mr. Thomas	male	NaN	1	
534	Cacic, Miss. Marija	female	30.0	0	
765	Hogeboom, Mrs. John C (Anna Andrews)	female	51.0	1	
811	Lester, Mr. James	male	39.0	0	
801	Collyer, Mrs. Harvey (Charlotte Annie Tate)	female	31.0	1	

	Parch	Ticket	Fare	Cabin	Embarked
678	6	CA 2144	46.9000	NaN	S
779	1	24160	211.3375	B3	S
881	0	349257	7.8958	NaN	S
364	0	370365	15.5000	NaN	Q
534	0	315084	8.6625	NaN	S
765	0	13502	77.9583	D11	S
811	0	A/4 48871	24.1500	NaN	S
801	1	C.A. 31921	26.2500	NaN	S

```

# 5. Shape
print(df.shape)

```

```

# 6. Column names
print(df.columns)

```

```

# 7. Data types
print(df.dtypes)

```

```

(891, 12)
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
PassengerId    int64
Survived       int64
Pclass         int64
Name           object
Sex            object
Age           float64
SibSp          int64
Parch          int64
Ticket         object
Fare           float64
Cabin          object
Embarked       object
dtype: object

```

```

# 8. Info
df.info()

```

```

# 9. Statistics
print(df.describe())

```

```

# 10. Total rows and columns
rows, columns = df.shape
print("Total Rows:", rows)
print("Total Columns:", columns)

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64

```

```

3  Name      891 non-null  object
4  Sex       891 non-null  object
5  Age       714 non-null  float64
6  SibSp     891 non-null  int64
7  Parch     891 non-null  int64
8  Ticket    891 non-null  object
9  Fare      891 non-null  float64
10 Cabin     204 non-null  object
11 Embarked  889 non-null  object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

```

	PassengerId	Survived	Pclass	Age	SibSp	\
count	891.000000	891.000000	891.000000	714.000000	891.000000	
mean	446.000000	0.383838	2.308642	29.699118	0.523008	
std	257.353842	0.486592	0.836071	14.526497	1.102743	
min	1.000000	0.000000	1.000000	0.420000	0.000000	
25%	223.500000	0.000000	2.000000	20.125000	0.000000	
50%	446.000000	0.000000	3.000000	28.000000	0.000000	
75%	668.500000	1.000000	3.000000	38.000000	1.000000	
max	891.000000	1.000000	3.000000	80.000000	8.000000	

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

Total Rows: 891
Total Columns: 12

```

# Missing Values Handling
# 11. Find total missing values in each column
print(df.isnull().sum())

# 12. Fill missing values in Age column with mean age
df["Age"] = df["Age"].fillna(df["Age"].mean())
print(df["Age"].isnull().sum())

# 13. Fill missing values in Embarked column with mode
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
print(df["Embarked"].isnull().sum())

# 14. Fill missing values in Cabin with 'Unknown'
df_cabin_unknown = df.copy()
df_cabin_unknown["Cabin"] = df_cabin_unknown["Cabin"].fillna("Unknown")
print(df_cabin_unknown["Cabin"].head())

```

```

PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age           177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin         687
Embarked       2
dtype: int64
0
0
0   Unknown
1    C85
2   Unknown
3    C123
4   Unknown
Name: Cabin, dtype: object

```

```

# column section
# 18. Select Name, Sex, and Age columns
print(df[["Name", "Sex", "Age"]])

# 19. Select Pclass, Fare, and Survived columns

```

```
print(df[["Pclass", "Fare", "Survived"]])
```

```
# 20. Display only Ticket and Cabin columns
print(df[["Ticket", "Cabin"]])
```

		Name	Sex	Age
0		Braund, Mr. Owen Harris	male	22.000000
1	Cummings, Mrs. John Bradley (Florence Briggs Th...		female	38.000000
2	Heikkinen, Miss. Laina		female	26.000000
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)		female	35.000000
4	Allen, Mr. William Henry		male	35.000000
..	...		...	...
886	Montvila, Rev. Juozas		male	27.000000
887	Graham, Miss. Margaret Edith		female	19.000000
888	Johnston, Miss. Catherine Helen "Carrie"		female	29.69118
889	Behr, Mr. Karl Howell		male	26.000000
890	Dooley, Mr. Patrick		male	32.000000

```
[891 rows x 3 columns]
```

	Pclass	Fare	Survived
0	3	7.2500	0
1	1	71.2833	1
2	3	7.9250	1
3	1	53.1000	1
4	3	8.0500	0
..	...	...	...
886	2	13.0000	0
887	1	30.0000	1
888	3	23.4500	0
889	1	30.0000	1
890	3	7.7500	0

```
[891 rows x 3 columns]
```

	Ticket	Cabin
0	A/5 21171	NaN
1	PC 17599	C85
2	STON/O2. 3101282	NaN
3	113803	C123
4	373450	NaN
..	...	...
886	211536	NaN
887	112053	B42
888	W./C. 6607	NaN
889	111369	C148
890	370376	NaN

```
[891 rows x 2 columns]
```

```
# Row Selection Using iloc[]
```

```
# 21. Display first 5 rows using iloc[]
print(df.iloc[:5])
```

```
# 22. Display rows 10 to 20
print(df.iloc[10:21])
```

```
# 23. Display rows 50 to 60 and columns Name, Age, Fare
print(df.iloc[50:61][["Name", "Age", "Fare"]])
```

```
# 24. Display last 5 rows using iloc[]
print(df.iloc[-5:])
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

		Name	Sex	Age	SibSp	\
0		Braund, Mr. Owen Harris	male	22.0	1	
1	Cummings, Mrs. John Bradley (Florence Briggs Th...		female	38.0	1	
2	Heikkinen, Miss. Laina		female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)		female	35.0	1	
4	Allen, Mr. William Henry		male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S

```

1      0      PC 17599 71.2833 C85      C
2      0 STON/O2. 3101282 7.9250 NaN      S
3      0      113803 53.1000 C123      S
4      0      373450 8.0500 NaN      S
   PassengerId  Survived  Pclass \
10           11         1      3
11           12         1      1
12           13         0      3
13           14         0      3
14           15         0      3
15           16         1      2
16           17         0      3
17           18         1      2
18           19         0      3
19           20         1      3
20           21         0      2

                                Name      Sex      Age \
10      Sandstrom, Miss. Marguerite Rut  female  4.000000
11      Bonnell, Miss. Elizabeth         female  58.000000
12      Saunderson, Mr. William Henry     male    20.000000
13      Andersson, Mr. Anders Johan       male    39.000000
14      Vestrom, Miss. Hulda Amanda Adolfina female  14.000000
15      Hewlett, Mrs. (Mary D Kingcome)   female  55.000000
16      Rice, Master. Eugene              male     2.000000
17      Williams, Mr. Charles Eugene       male   29.699118
18 Vander Planke, Mrs. Julius (Emelia Maria Vande... female  31.000000
19      Masselmani, Mrs. Fatima            female  29.699118
20      Fynney, Mr. Joseph J               male   35.000000

   SibSp  Parch  Ticket  Fare Cabin Embarked
10      1      1  PP 9549  16.7000  G6      S
11      0      0  113783  26.5500 C103      S
12      0      0  A/5. 2151  8.0500  NaN      S
13      1      5  347082  31.2750  NaN      S
14      0      0  350406  7.8542  NaN      S
15      0      0  248706  16.0000  NaN      S
16      4      1  382652  29.1250  NaN      Q
17      0      0  244373  13.0000  NaN      S
18      1      0  345763  18.0000  NaN      S
19      0      0   2649   7.2250  NaN      C
20      0      0  239865  26.0000  NaN      S

```

```

# Row Selection Using loc[]

# 25. Display passengers whose age is greater than 60
print(df.loc[df["Age"] > 60])

# 26. Display passengers whose fare is greater than 100
print(df.loc[df["Fare"] > 100])

# 27. Display passengers whose sex is female
print(df.loc[df["Sex"] == "female"])

# 28. Display passengers who survived
print(df.loc[df["Survived"] == 1])

# 29. Display passengers in First Class
print(df.loc[df["Pclass"] == 1])

```

```

   PassengerId  Survived  Pclass                                Name \
33           34         0      2      Wheadon, Mr. Edward H
54           55         0      1      Ostby, Mr. Engelhart Cornelius
96           97         0      1      Goldschmidt, Mr. George B
116          117         0      3      Connors, Mr. Patrick
170          171         0      1      Van der hoef, Mr. Wyckoff
252          253         0      1      Stead, Mr. William Thomas
275          276         1      1      Andrews, Miss. Kornelia Theodosia
280          281         0      3      Duane, Mr. Frank
326          327         0      3      Nysveen, Mr. Johan Hansen
438          439         0      1      Fortune, Mr. Mark
456          457         0      1      Millet, Mr. Francis Davis
483          484         1      3      Turkula, Mrs. (Hedwig)
493          494         0      1      Artagaveytia, Mr. Ramon
545          546         0      1      Nicholson, Mr. Arthur Ernest
555          556         0      1      Wright, Mr. George
570          571         1      2      Harris, Mr. George
625          626         0      1      Sutton, Mr. Frederick

```

630	631	1	1	Barkworth, Mr. Algernon Henry Wilson
672	673	0	2	Mitchell, Mr. Henry Michael
745	746	0	1	Crosby, Capt. Edward Gifford
829	830	1	1	Stone, Mrs. George Nelson (Martha Evelyn)
851	852	0	3	Svensson, Mr. Johan

	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
33	male	66.0	0	0	C.A. 24579	10.5000	NaN	S
54	male	65.0	0	1	113509	61.9792	B30	C
96	male	71.0	0	0	PC 17754	34.6542	A5	C
116	male	70.5	0	0	370369	7.7500	NaN	Q
170	male	61.0	0	0	111240	33.5000	B19	S
252	male	62.0	0	0	113514	26.5500	C87	S
275	female	63.0	1	0	13502	77.9583	D7	S
280	male	65.0	0	0	336439	7.7500	NaN	Q
326	male	61.0	0	0	345364	6.2375	NaN	S
438	male	64.0	1	4	19950	263.0000	C23 C25 C27	S
456	male	65.0	0	0	13509	26.5500	E38	S
483	female	63.0	0	0	4134	9.5875	NaN	S
493	male	71.0	0	0	PC 17609	49.5042	NaN	C
545	male	64.0	0	0	693	26.0000	NaN	S
555	male	62.0	0	0	113807	26.5500	NaN	S
570	male	62.0	0	0	S.W./PP 752	10.5000	NaN	S
625	male	61.0	0	0	36963	32.3208	D50	S
630	male	80.0	0	0	27042	30.0000	A23	S
672	male	70.0	0	0	C.A. 24580	10.5000	NaN	S
745	male	70.0	1	1	WE/P 5735	71.0000	B22	S
829	female	62.0	0	0	113572	80.0000	B28	S
851	male	74.0	0	0	347060	7.7750	NaN	S

PassengerId	Survived	Pclass	\
27	0	1	
31	1	1	
88	1	1	
118	0	1	
195	1	1	
215	1	1	
258	1	1	
268	1	1	
269	1	1	
207	0	1	

```
# Unique and Frequency Analysis
```

```
# 30. Find all unique values in the Sex column
print(df["Sex"].unique())
```

```
# 31. Count number of male and female passengers
print(df["Sex"].value_counts())
```

```
# 32. Find all unique values in the Embarked column
print(df["Embarked"].unique())
```

```
# 33. Count number of passengers in each passenger class
print(df["Pclass"].value_counts())
```

```
# 34. Count how many passengers survived and did not survive
print(df["Survived"].value_counts())
```

```
['male' 'female']
Sex
male      577
female    314
Name: count, dtype: int64
['S' 'C' 'Q']
Pclass
3      491
1      216
2      184
Name: count, dtype: int64
Survived
0      549
1      342
Name: count, dtype: int64
```

```
# Statistical Analysis
```

```
# 35. Find the mean age of passengers
print(df["Age"].mean())
```

```
# 36. Find the median fare
print(df["Fare"].median())

# 37. Find the mode of the Embarked column
print(df["Embarked"].mode()[0])

# 38. Find average fare paid by female passengers
print(df.loc[df["Sex"] == "female", "Fare"].mean())

# 39. Find median age of passengers who survived
print(df.loc[df["Survived"] == 1, "Age"].median())

# 40. Find mean fare of first-class passengers
print(df.loc[df["Pclass"] == 1, "Fare"].mean())
```

```
29.69911764705882
14.4542
S
44.47981783439491
29.69911764705882
84.1546875
```

```
# Using apply()

# 46. Convert all passenger names to uppercase
df["Name_upper"] = df["Name"].apply(lambda x: x.upper())
print(df["Name_upper"].head())

# 47. Find the length of each passenger name
df["Name_length"] = df["Name"].apply(lambda x: len(x))
print(df["Name_length"].head())

# 48. Round the Fare column to two decimal places
df["Fare_rounded"] = df["Fare"].apply(lambda x: round(x, 2))
print(df["Fare_rounded"].head())

# 49. Convert Sex column to title case (Male, Female)
df["Sex_title"] = df["Sex"].apply(lambda x: x.title())
print(df["Sex_title"].head())
```

```
0          BRAUND, MR. OWEN HARRIS
1  CUMINGS, MRS. JOHN BRADLEY (FLORENCE BRIGGS TH...
2          HEIKKINEN, MISS. LAINA
3  FUTRELLE, MRS. JACQUES HEATH (LILY MAY PEEL)
4          ALLEN, MR. WILLIAM HENRY
Name: Name_upper, dtype: object
0    23
1    51
2    22
3    44
4    24
Name: Name_length, dtype: int64
0     7.25
1    71.28
2     7.92
3    53.10
4     8.05
Name: Fare_rounded, dtype: float64
0    Male
1  Female
2  Female
3  Female
4    Male
Name: Sex_title, dtype: object
```

```
# Using map()

# 50. male -> M, female -> F
df["Sex_mapped"] = df["Sex"].map({"male": "M", "female": "F"})
print(df["Sex_mapped"].head())
```

```
# 51. 0 -> Did Not Survive, 1 -> Survived
df["Survival_status"] = df["Survived"].map({0: "Did Not Survive", 1: "Survived"})
print(df["Survival_status"].head())

# 52. Pclass mapping
df["Class_mapped"] = df["Pclass"].map({
    1: "First Class",
    2: "Second Class",
    3: "Third Class"
})
print(df["Class_mapped"].head())
```

```
0    M
1    F
2    F
3    F
4    M
Name: Sex_mapped, dtype: object
0    Did Not Survive
1         Survived
2         Survived
3         Survived
4    Did Not Survive
Name: Survival_status, dtype: object
0    Third Class
1    First Class
2    Third Class
3    First Class
4    Third Class
Name: Class_mapped, dtype: object
```